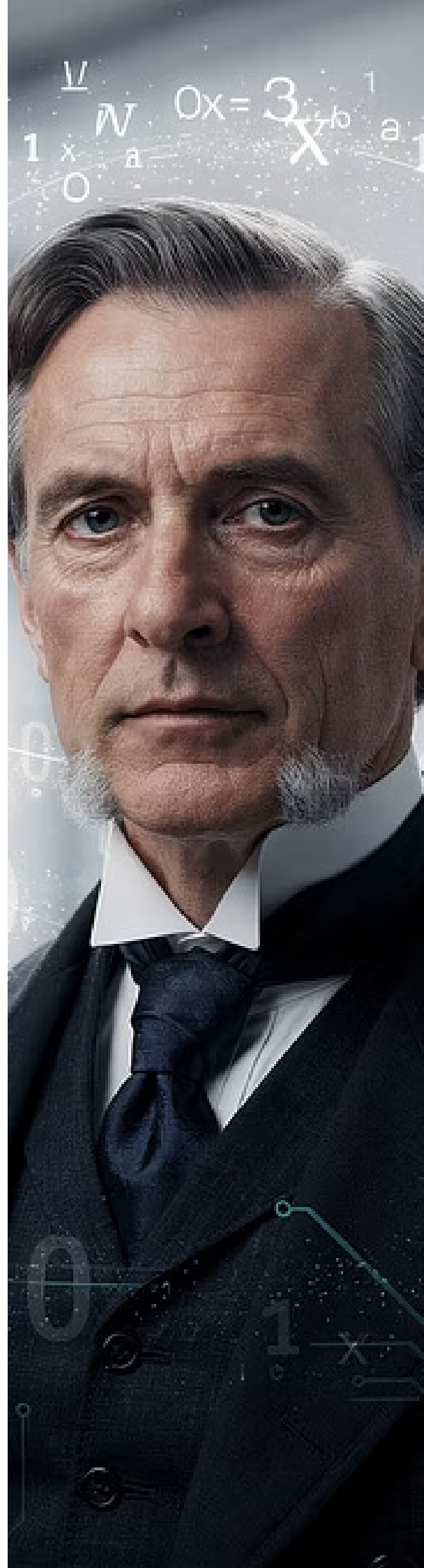


Álgebra de Boole, Funciones Booleanas y Compuertas Lógicas

Si la lógica proposicional es el "lenguaje" con el que razonamos, el Álgebra de Boole es la estructura matemática que permite calcular y simplificar esos razonamientos. Fue desarrollada por George Boole a mediados del siglo XIX, mucho antes de que existieran las computadoras digitales.

Para un técnico en software, dominar esta álgebra no es opcional. Es lo que te permitirá tomar una condición compleja de 10 líneas de código y reducirla a una sola línea eficiente, elegante y libre de errores.



¿Qué es el Álgebra de Boole?

Boole tuvo una visión genial: la lógica humana puede reducirse a un sistema algebraico simple, similar a la aritmética que usas para sumar números, pero con reglas ligeramente distintas y solo dos valores posibles: **0** y **1**.

El Álgebra de Boole se define formalmente como una **estructura matemática** sobre un conjunto $B = \{0, 1\}$ regida por los **Axiomas de Huntington**. A diferencia de la aritmética convencional, se basa en la lógica de clases y la teoría de conjuntos.

El conjunto base

Solo dos valores posibles: **0** (Falso) y **1** (Verdadero).

Operaciones binarias

Suma Lógica (+) y Producto Lógico (·) como operaciones principales.

Operación unaria

El **complemento** (barra superior o apóstrofe), equivalente al NOT lógico.

❏ Es crucial que no confundas estos símbolos con la suma y multiplicación de números reales. Las reglas son distintas y solo operan sobre el conjunto $B = \{0, 1\}$.

Contexto Histórico

George Boole publicó su obra fundamental "An Investigation of the Laws of Thought" en 1854. Décadas más tarde, en 1937, Claude Shannon demostró que el álgebra de Boole podía aplicarse de manera brillante al diseño de circuitos eléctricos, sentando las bases fundamentales de la electrónica digital moderna y la computación que utilizamos hoy.

Característica	Aritmética Convencional	Álgebra de Boole
Conjunto de valores	$\mathbb{R}$ (infinitos)	$\{0, 1\}$ (solo dos)
Suma	$1+1=2$	$1+1=1$
Producto	$2 \cdot 2=4$	$1 \cdot 1=1$
Complemento	No existe	$\bar{A}$ (NOT)
Uso principal	Cálculo numérico	Lógica y circuitos

❏ **Regla de oro:** en Boole, $1+1=1$ (no 2). Si dos condiciones verdaderas se unen con OR, el resultado sigue siendo Verdadero, no 'doble verdadero'.

Definición Formal: Operaciones y Operadores Lógicos

El Álgebra de Boole se define sobre un conjunto $B = \{0, 1\}$ y dos operaciones binarias principales: la **Suma Lógica (+)** y el **Producto Lógico (·)**. Existe además una operación unaria: el **complemento**.

La relación directa con los operadores lógicos es la siguiente:

Símbolo Booleano	Operador Lógico	Valor 0	Valor 1
Suma (+)	OR ($\vee$)	Falso	Verdadero
Producto (·)	AND ($\wedge$)	Falso	Verdadero
Complemento ($\neg$ / barra)	NOT ($\neg$)	Falso	Verdadero

Así, el **0** representa Falso (False), el **1** representa Verdadero (True), la **Suma (+)** equivale al **OR ($\vee$)**, el **Producto (·)** equivale al **AND ($\wedge$)**, y el complemento (o barra superior, o apóstrofe) equivale al **NOT ($\neg$)**.

Tablas de Verdad de las Operaciones Básicas

OR (A+B)

A,B	Res
0,0	0
0,1	1
1,0	1
1,1	1

AND (A·B)

A,B	Res
0,0	0
0,1	0
1,0	0
1,1	1

NOT (Ā)

A	Res
0	1
1	0

Ejemplo en Código (R)

```
A = TRUE # 1
B = FALSE # 0

# OR → Suma Lógica (+)
print(A | B) # TRUE (1)

# AND → Producto Lógico (·)
print(A & B) # FALSE (0)

# NOT → Complemento (¬)
print(!A) # FALSE (0)
```

☐ En R, los operadores `|`, `&` y `!` implementan directamente las operaciones del Álgebra de Boole. TRUE equivale a 1 y FALSE equivale a 0.

Leyes del Álgebra de Boole y Compuertas Lógicas

Los postulados del Álgebra de Boole son las reglas fundamentales que rigen todas las operaciones. Se dividen en dos grupos: Postulados de Identidad y Dominación (#1 y #2), e Idempotencia y Complemento (#3 y #4). Cada postulado tiene una forma para la Suma Lógica (+) y una forma dual para el Producto Lógico (·).

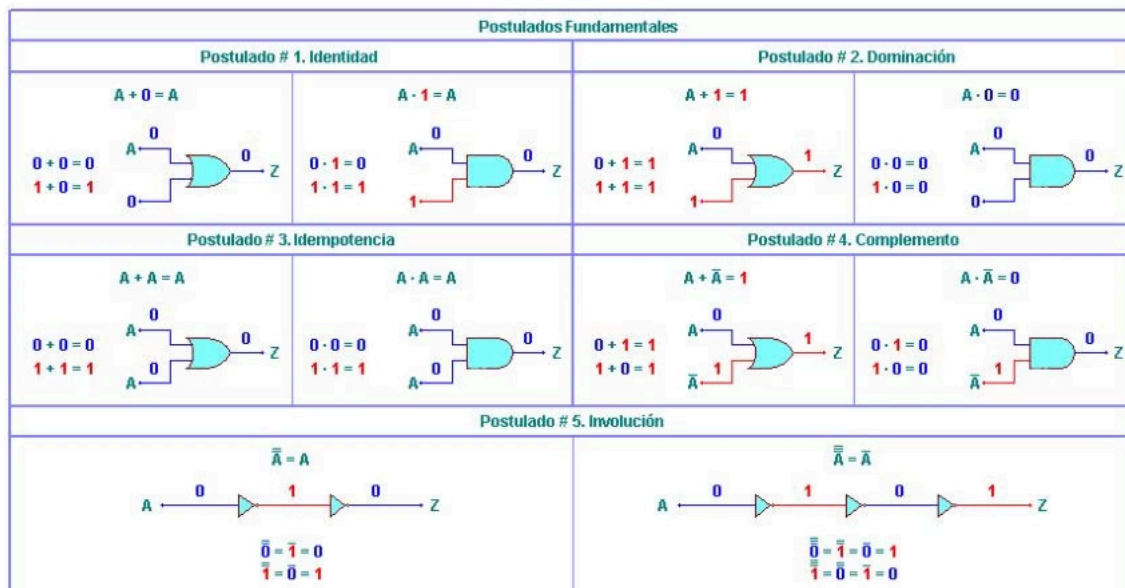


Figura 1. Leyes del Álgebra de Boole y compuertas lógicas. Fuente: Wilaelectronica. (2019, 31 octubre). *Algebra de Boole* [Gráfico]. <https://wilaelectronica.blogspot.com/2019/10/algebra-de-boole.html>

La figura ilustra los postulados de Identidad, Dominación, Idempotencia, Complemento e Involución mediante compuertas lógicas OR y AND, mostrando sus tablas de verdad correspondientes.

Postulado #1 — Identidad (OR)

$A + 0 = A$ y $A + 1 = 1$

Postulado #1 — Identidad (AND)

$A \cdot 1 = A$ y $A \cdot 0 = 0$

Postulado #2 — Dominación (OR)

Si uno de los operandos es 1, el resultado de OR siempre es 1. Ejemplo: $0+1=1$, $1+1=1$

Postulado #2 — Dominación (AND)

Si uno de los operandos es 0, el resultado de AND siempre es 0. Ejemplo: $0 \cdot 0 = 0$, $1 \cdot 0 = 0$



Interpretación práctica: el 0 es el elemento neutro del OR (no cambia nada), y el 1 es el elemento neutro del AND (no cambia nada). El 1 es el elemento absorbente del OR, y el 0 es el elemento absorbente del AND.

Postulados #3 y #4: Idempotencia y Complemento

Postulado #3 — Idempotencia

$$A + A = A$$

$$A \cdot A = A$$

Ejemplos: $0 + 0 = 0$, $1 + 1 = 1$, $0 \cdot 0 = 0$, $1 \cdot 1 = 1$

Interpretación: Si verificas dos veces si un usuario está activo (if active && active), es lo mismo que verificarlo una sola vez. En aritmética normal $x + x = 2x$, pero en Boole, la redundancia no suma valor.

Postulado #4 — Complemento

$$A + \neg A = 1$$

Algo es cierto o no lo es, no hay término medio. **Principio del Tercero Excluido.**

$$A \cdot \neg A = 0$$

Algo no puede ser cierto y falso a la vez. **Principio de No Contradicción.**

Ejemplos: $0 + 1 = 1$, $1 + 0 = 1$, $0 \cdot 1 = 0$, $1 \cdot 0 = 0$

Postulado #5 — Involución

$$\overline{\overline{A}} = A$$

$$\overline{0} = 1 \quad \overline{1} = 0$$

$$\overline{\overline{\overline{A}}} = \overline{A}$$

$$\overline{\overline{0}} = 1 \quad \overline{\overline{1}} = 0$$

Verificación con Tablas de Verdad

Idempotencia OR

A	A+A	Res
0	0+0	0
1	1+1	1

Complemento

A	$\overline{A}$	$A + \overline{A}$	$A \cdot \overline{A}$
0	1	1	0
1	0	1	0

Involución

A	$\overline{\overline{A}}$	$\overline{\overline{\overline{A}}}$
0	1	0
1	0	1

Aplicación en Programación

```
A = TRUE

# Idempotencia: A or A == A
print(A | A) # TRUE (igual que A)
print(A & A) # TRUE (igual que A)

# Complemento: A or not A == True
print(A | !A) # TRUE (siempre)
print(A & !A) # FALSE (siempre)

# Involución: not not A == A
print(!!A) # TRUE (igual que A)
```

Propiedades: Conmutativa, Asociativa y Distributiva

Además de los postulados, el Álgebra de Boole cumple con tres propiedades fundamentales que permiten reorganizar y simplificar expresiones.

1	2	3
Propiedad Conmutativa $A + B = B + A$ $A \cdot B = B \cdot A$ El orden de los operandos no afecta el resultado.	Propiedad Asociativa $A + (B + C) = (A + B) + C$ $A \cdot (B \cdot C) = (A \cdot B) \cdot C$ El agrupamiento de operandos no afecta el resultado.	Propiedad Distributiva $A + B \cdot C = (A + B) \cdot (A + C)$ $A \cdot (B + C) = A \cdot B + A \cdot C$ Especialmente notable: la distributividad de la suma respecto al producto $A + (B \cdot C) = (A + B) \cdot (A + C)$, que no existe en la aritmética convencional.

Ejemplos Numéricos Paso a Paso

Conmutativa

A=1, B=0

$A+B = 1+0 = 1$

$B+A = 0+1 = 1$

Resultado: iguales ✓

$A \cdot B = 1 \cdot 0 = 0$

$B \cdot A = 0 \cdot 1 = 0$

Resultado: iguales ✓

Asociativa

A=1, B=0, C=1

$A+(B+C) = 1+(0+1) = 1+1 = 1$

$(A+B)+C = (1+0)+1 = 1+1 = 1$

Resultado: iguales ✓

Distributiva especial

A=0, B=1, C=1

$A+(B \cdot C) = 0+(1 \cdot 1) = 0+1 = 1$

$(A+B) \cdot (A+C) = (0+1) \cdot (0+1) = 1 \cdot 1 = 1$

Resultado: iguales ✓

❏ ¡Ojo! La propiedad distributiva del OR sobre el AND — $A+(B \cdot C) = (A+B) \cdot (A+C)$ — NO existe en la aritmética convencional. Es exclusiva del Álgebra de Boole y suele causar errores si se olvida.

Aplicación en Código

```
A <- TRUE; B <- FALSE; C <- TRUE

# Conmutativa
print(A | B == B | A) # TRUE
print(A & B == B & A) # TRUE

# Asociativa
print(A | (B | C) == (A | B) | C) # TRUE

# Distributiva
print(A | (B & C) == (A | B) & (A | C)) # TRUE
```

Propiedades del Álgebra de Boole y Compuertas Lógicas

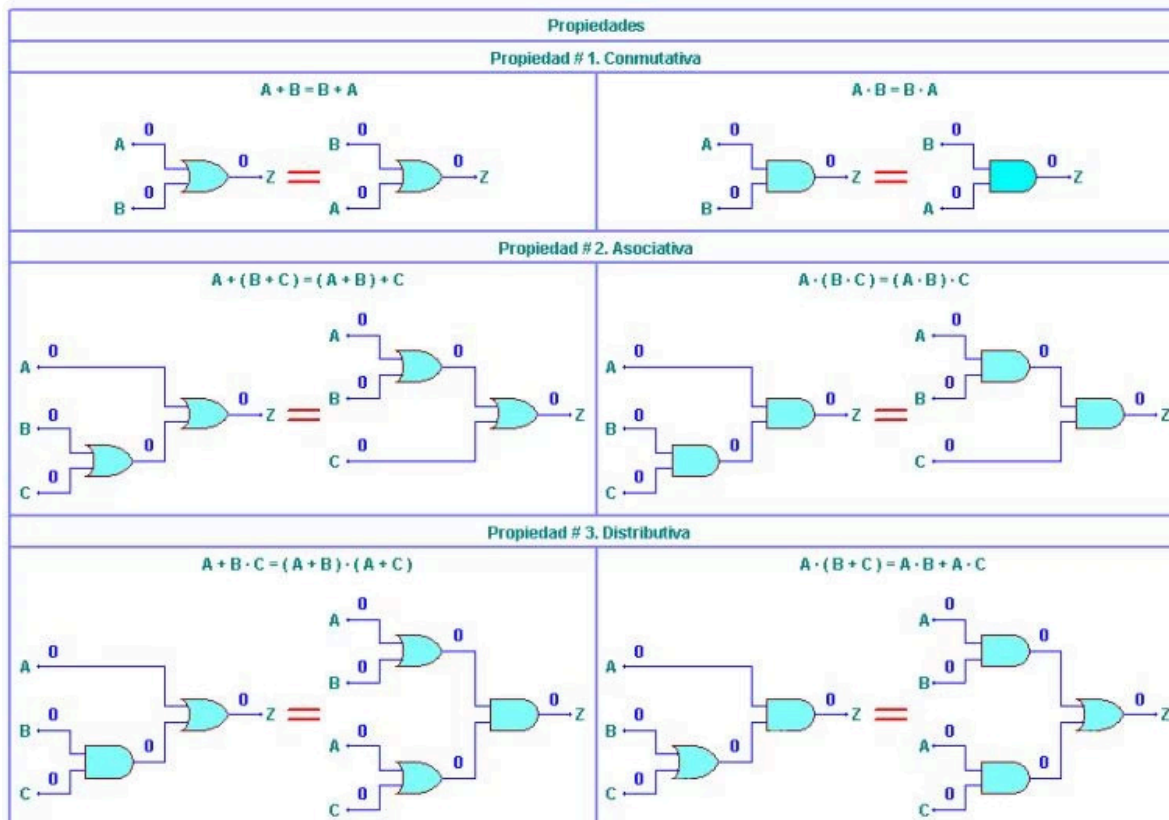


Figura 2. Leyes del Álgebra de Boole y compuertas lógicas. Fuente: Wilaelectronica. (2019, 31 octubre). *Algebra de Boole* [Gráfico]. <https://wilaelectronica.blogspot.com/2019/10/algebra-de-boole.html>

La figura ilustra las propiedades Conmutativa, Asociativa y Distributiva del Álgebra de Boole mediante compuertas lógicas OR y AND, mostrando la equivalencia entre las distintas formas de agrupar y distribuir los operandos.

La figura muestra cómo las propiedades Conmutativa, Asociativa y Distributiva se implementan físicamente con compuertas lógicas. Observa que reorganizar las compuertas no cambia la salida del circuito.

Resumen Visual de las Propiedades

Conmutativa $A + B = B + A$ $A \cdot B = B \cdot A$ El orden no importa	Asociativa $A + (B + C) = (A + B) + C$ $A \cdot (B \cdot C) = (A \cdot B) \cdot C$ El agrupamiento no importa	Distributiva $A \cdot (B + C) = A \cdot B + A \cdot C$ $A + (B \cdot C) = (A + B) \cdot (A + C)$ Exclusiva de Boole: el OR también distribuye sobre el AND
---	---	--

- Estas propiedades son la base para simplificar circuitos lógicos complejos. Un circuito con menos compuertas consume menos energía y es más rápido.

Teoremas de Simplificación: Absorción

Además de los postulados básicos, el Álgebra de Boole cuenta con teoremas de simplificación que permiten reducir expresiones complejas. Entre los más importantes se encuentran las leyes de **Absorción**.

Absorción (forma suma)

$$A + A \cdot B = A$$

Un término "absorbe" a otro que lo contiene como factor. La expresión se reduce al término más simple.

Absorción (forma producto)

$$A \cdot (A + B) = A$$

De igual forma, el producto de A con una suma que contiene A se reduce simplemente a A.

Absorción con complemento (suma)

$$A + \bar{A}B = A + B$$

Identidad #11: permite eliminar el complemento del primer término cuando aparece como factor del segundo.

Absorción con complemento (producto)

$$A \cdot (\bar{A} + B) = A \cdot B$$

Identidad #15: el complemento del primer factor se elimina al distribuir el producto.

- ☐ Estas leyes de absorción son fundamentales para la **simplificación de circuitos lógicos** y la optimización de condiciones en código. Permiten eliminar términos redundantes sin alterar el resultado lógico.

Demostración Algebraica de la Absorción

Demostración: $A + A \cdot B = A$

Paso 1: $A + A \cdot B = A \cdot 1 + A \cdot B$ (Identidad: $A \cdot 1 = A$)

Paso 2: $= A \cdot (1 + B)$ (Distributiva)

Paso 3: $= A \cdot 1$ (Dominación: $1 + B = 1$)

Paso 4: $= A$ (Identidad) ✓

Demostración: $A \cdot (A + B) = A$

Paso 1: $A \cdot (A + B) = A \cdot A + A \cdot B$ (Distributiva)

Paso 2: $= A + A \cdot B$ (Idempotencia: $A \cdot A = A$)

Paso 3: $= A$ (Absorción suma, ya demostrada) ✓

Ejemplo Práctico en Código

```
# Simplificación de condición compleja
usuario_activo = TRUE
tiene_permiso = FALSE

# Sin simplificar (redundante):
if (usuario_activo | (usuario_activo & tiene_permiso)) {
  print("Acceso permitido")
}

# Simplificado con Absorción (A + A·B = A):
if (usuario_activo) {
  print("Acceso permitido") # ¡Mismo resultado, código más limpio!
}
```

- ☐ La absorción es tu mejor aliada para limpiar condiciones redundantes en código. Si ves 'if A or (A and B)', simplifica a 'if A'.

Leyes de De Morgan: ¡Vitales para Programar!

Las Leyes de De Morgan nos enseñan cómo negar paréntesis. Son esenciales para transformar expresiones lógicas y evitar errores graves en programación.

Primera Ley de De Morgan

$$\overline{A + B} = \bar{A} \cdot \bar{B}$$

Lectura: "La negación de una disyunción es la conjunción de las negaciones."

Demostración algebraica: $\bar{0} + \bar{0} = 1$, $0 + 1 = 1$, $1 + 0 = 1$, $1 + 1 = 0$

Segunda Ley de De Morgan

$$\overline{A \cdot B} = \bar{A} + \bar{B}$$

Lectura: "La negación de una conjunción es la disyunción de las negaciones."

Demostración algebraica: $\bar{0} \cdot \bar{0} = 1$, $0 \cdot 1 = 0$, $1 \cdot 0 = 0$, $1 \cdot 1 = 1$

También se expresan en la forma equivalente:

$$\bar{A} + \bar{B} = \overline{A \cdot B}$$

$$\bar{A} \cdot \bar{B} = \overline{A + B}$$

Tablas de Verdad: Verificación de De Morgan

Primera Ley: NOT(A+B) = NOT(A)·NOT(B)

A	B	A+B	NOT(A+B)	NOT(A)	NOT(B)	NOT(A)·NOT(B)
0	0	0	1	1	1	1
0	1	1	0	1	0	0
1	0	1	0	0	1	0
1	1	1	0	0	0	0

Nota: Las columnas NOT(A+B) y NOT(A)·NOT(B) son idénticas ✓

Segunda Ley: NOT(A·B) = NOT(A)+NOT(B)

A	B	A·B	NOT(A·B)	NOT(A)	NOT(B)	NOT(A)+NOT(B)
0	0	0	1	1	1	1
0	1	0	1	1	0	1
1	0	0	1	0	1	1
1	1	1	0	0	0	0

Nota: Las columnas NOT(A·B) y NOT(A)+NOT(B) son idénticas ✓

☐ Error clásico en programación: $!(A \mid B) \neq !A \mid !B$. Debes cambiar el $\mid$ por $\&$ al distribuir el $!$. De Morgan te salva de bugs lógicos difíciles de detectar.

Teorema de De Morgan y Compuertas Lógicas

Las Leyes de De Morgan tienen una representación directa en compuertas lógicas. Una compuerta NOR (NOT-OR) es equivalente a una compuerta AND con entradas invertidas. Una compuerta NAND (NOT-AND) es equivalente a una compuerta OR con entradas invertidas.

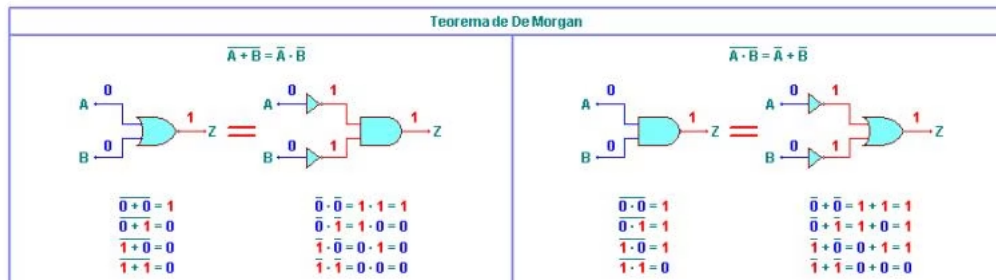


Figura 3. Leyes del Álgebra de Boole y compuertas lógicas. Fuente: Wilaebela electronica. (2019, 31 octubre). *Algebra de Boole* [Gráfico]. <https://wilaeblelectronica.blogspot.com/2019/10/algebra-de-boole.html>

La figura ilustra el Teorema de De Morgan mediante compuertas lógicas: una compuerta OR con entradas invertidas (burbujas) es equivalente a una compuerta AND con salida invertida (burbuja), y viceversa.

Equivalencias de Compuertas según De Morgan

Compuerta NOR

$$\overline{A+B} = \bar{A} \cdot \bar{B}$$

OR seguido de NOT = AND con entradas negadas

Compuerta NAND

$$\overline{A \cdot B} = \bar{A} + \bar{B}$$

AND seguido de NOT = OR con entradas negadas

Universalidad NAND

Con solo compuertas NAND se puede construir cualquier circuito lógico (NOT, AND, OR). Es la compuerta más usada en hardware.

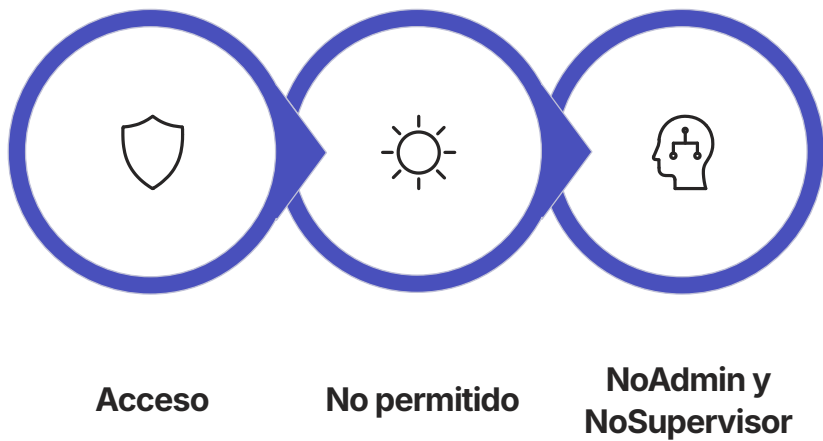
Universalidad NOR

Con solo compuertas NOR también se puede construir cualquier circuito lógico. Alternativa a NAND en diseño digital.

- ❏ Las compuertas NAND y NOR son llamadas 'compuertas universales' porque con ellas solas se puede implementar cualquier función lógica. Esto simplifica enormemente la fabricación de chips.

Aplicación Práctica de De Morgan en Programación

Imagina que quieres validar que un usuario **NO** tenga acceso. La condición de acceso es "Ser admin O ser supervisor".



Aplicando De Morgan paso a paso:

1. **Condición Acceso:** (Admin | Supervisor)
2. **Bloqueo:** !(Admin | Supervisor)
3. **Aplicando De Morgan:** (!Admin) & (!Supervisor)

Si intentas programar `if (!Admin | !Supervisor)` estarías cometiendo un error lógico grave. De Morgan te dice que debes **cambiar el OR por un AND** al distribuir el NO.

La importancia de estas leyes en la transformación de expresiones lógicas radica en que permiten reescribir cualquier expresión negada de forma equivalente, facilitando tanto la simplificación algebraica como la implementación correcta en código.

Más Ejemplos de De Morgan en Código

```
# Ejemplo 1 - Validación de formulario
campo_vacio <- FALSE
longitud_invalida <- TRUE
# Negación INCORRECTA: !campo_vacio | !longitud_invalida
# Negación CORRECTA con De Morgan: !(campo_vacio | longitud_invalida)
# que equivale a: !campo_vacio & !longitud_invalida -> FALSE

# Ejemplo 2 - Control de acceso
es_admin <- FALSE
es_supervisor <- FALSE
# !(admin | supervisor) = !admin & !supervisor
sin_privilegios <- !es_admin & !es_supervisor # -> TRUE
```

Generalización de De Morgan

Para n variables (OR)

$$\overline{A_1 + A_2 + \dots + A_n} = \bar{A}_1 \cdot \bar{A}_2 \cdot \dots \cdot \bar{A}_n$$

La negación de una suma lógica de n variables es el producto de sus complementos.

Para n variables (AND)

$$\overline{A_1 \cdot A_2 \cdot \dots \cdot A_n} = \bar{A}_1 + \bar{A}_2 + \dots + \bar{A}_n$$

La negación de un producto lógico de n variables es la suma de sus complementos.

Principio de Dualidad

Cualquier identidad algebraica booleana sigue siendo válida si intercambias los operadores + por · y los elementos neutros 0 por 1.



Identidad original
 $A \cdot 1 = A$
Producto Lógico con elemento neutro 1.



Intercambio dual
Cambia (·) → (+) y 1 → 0
Aplica el principio de dualidad.



Identidad dual
 $A + 0 = A$
Suma Lógica con elemento neutro 0. ¡Igualmente válida!

Esto reduce a la mitad la cantidad de fórmulas que necesitas memorizar. Si sabes que $A \cdot 1 = A$, por dualidad sabes automáticamente que $A + 0 = A$. Toda identidad booleana genera una "gemela" al intercambiar operadores y elementos neutros.



El Principio de Dualidad es una herramienta poderosa: cada teorema que demuestras te da automáticamente otro teorema "gratis", simplemente intercambiando (+) ↔ (·) y 0 ↔ 1.

Tabla de Dualidad: Pares de Identidades

#	Identidad Original	# Dual	Identidad Dual
1	$A + 0 = A$	2	$A \cdot 1 = A$
3	$A + 1 = 1$	4	$A \cdot 0 = 0$
5	$A + A = A$	6	$A \cdot A = A$
7	$A + \bar{A} = 1$	8	$A \cdot \bar{A} = 0$
9	$A + (B \cdot C) = (A + B) \cdot (A + C)$	10	$A \cdot (B + C) = (A \cdot B) + (A \cdot C)$
11	$A \cdot (B + C) = (A \cdot B) + (A \cdot C)$	12	$A + (B \cdot C) = (A + B) \cdot (A + C)$

Regla de Aplicación

01

Paso 1
Toma cualquier identidad booleana válida

02

Paso 2
Intercambia todos los (+) por (·) y todos los (·) por (+). Cambia todos los 0 por 1 y todos los 1 por 0

03

Paso 3
La nueva expresión obtenida también es una identidad válida (su dual)



Truco de estudio: aprende solo la mitad de las identidades. La otra mitad la obtienes gratis aplicando dualidad. Esto reduce a la mitad el tiempo de memorización.

Identidades Básicas del Álgebra de Boole

A continuación se presentan las 16 identidades básicas del Álgebra de Boole, que sintetizan todos los postulados, propiedades y teoremas vistos:

#	Identidad	#	Identidad
1	$A + \bar{A} = 1$	9	$\bar{\bar{A}} = A$
2	$A + 0 = A$	10	$A + AB = A$
3	$A + 1 = 1$	11	$A + \bar{A}B = A + B$
4	$A + A = A$	12	$\bar{A} + \bar{B} = \overline{A \cdot B}$
5	$A \cdot \bar{A} = 0$	13	$\bar{\bar{\bar{A}}} = \bar{A}$
6	$A \cdot 0 = 0$	14	$A \cdot (A + B) = A$
7	$A \cdot 1 = A$	15	$A \cdot (\bar{A} + B) = A \cdot B$
8	$A \cdot A = A$	16	$\bar{A} \cdot \bar{B} = \overline{A + B}$

Figura 4. Leyes del Álgebra de Boole y compuertas lógicas. Fuente: Wilaebela electronica. (2019, 31 octubre). *Algebra de Boole* [Gráfico]. <https://wilaeblelectronica.blogspot.com/2019/10/algebra-de-boole.html>

Clasificación de las Identidades por Tipo

Complemento (#1, #5)
 $A + \bar{A} = 1, A \cdot \bar{A} = 0$
Principio del Tercero Excluido y No Contradicción

Identidad y Dominación (#2, #3, #6, #7)
 $A + 0 = A, A + 1 = 1, A \cdot 0 = 0, A \cdot 1 = A$
Elementos neutros y absorbentes

Idempotencia e Involución (#4, #8, #9, #13)
 $A + A = A, A \cdot A = A, \bar{\bar{A}} = A$
Redundancia y doble negación

Absorción (#10, #11, #14, #15)
 $A + AB = A, A \cdot (A + B) = A$
Eliminación de términos redundantes

De Morgan (#12, #16)
 $\bar{A} + \bar{B} = \text{NOT}(A \cdot B), \bar{A} \cdot \bar{B} = \text{NOT}(A + B)$
Transformación de expresiones negadas

Ejercicio de Simplificación

Simplificar: $F = A \cdot B + A \cdot \bar{B} + \bar{A} \cdot B$

- Paso 1: $A \cdot B + A \cdot \bar{B} = A \cdot (B + \bar{B}) = A \cdot 1 = A$ (Distributiva + Complemento + Identidad)
- Paso 2: $F = A + \bar{A} \cdot B$ (resultado del paso 1)
- Paso 3: $A + \bar{A} \cdot B = A + B$ (Identidad #11: $A + \bar{A}B = A + B$)
- Resultado final: $F = A + B$ ✓

📌 Estas 16 identidades son tu caja de herramientas completa. Cualquier simplificación de expresión booleana se resuelve aplicando estas identidades en el orden correcto.

Resumen: ¿Por Qué Dominar el Álgebra de Boole?

Para un técnico en software, dominar esta álgebra no es opcional. Es lo que te permitirá tomar una condición compleja de 10 líneas de código y reducirla a una sola línea eficiente, elegante y libre de errores.



Definición Formal

Estructura matemática sobre $B = \{0,1\}$ con operaciones binarias (+) y (·) y unaria (complemento), relacionadas con OR, AND y NOT.



Teoremas de Simplificación

Leyes de Absorción ($A + A \cdot B = A$) y otras identidades que permiten reducir expresiones complejas.



Principio de Dualidad

Toda identidad booleana genera una "gemela" al intercambiar (+) ↔ (·) y 0 ↔ 1. Reduce a la mitad las fórmulas a memorizar.



Postulados y Axiomas

Identidad, Dominación, Idempotencia, Complemento, Involución, Conmutativa, Asociativa y Distributiva (incluyendo $A + (B \cdot C) = (A + B) \cdot (A + C)$).



Leyes de De Morgan

Transformación de expresiones negadas: $\overline{A \cdot B} = \bar{A} + \bar{B}$ y $\overline{A + B} = \bar{A} \cdot \bar{B}$. Vitales para programar correctamente.